to return. Similarly, we can use `_source_excludes` to exclude the fields from the response; no surprise there.

We may need to enhance our `movie` documents for this example, as the current document has no more than two fields. Let's add a few extra fields for a third movie (*The Shawshank Redemption*).

Listing 5.14 New `movie` document with additional attributes

```
PUT movies/_doc/3                          <-- Indexing a new document
{
  "title":"The Shawshank Redemption",
  "synopsis":"Two imprisoned men bond ..",
  "rating":9.3,                            <-- New field: rating attribute
  "certificate":"15",                      <-- New field: certificate attribute
  "genre":"drama",
  "actors":["Morgan Freeman","Tim Robbins"]
}
```

Once this document is indexed, we can experiment with fields that should or should not be returned in the response.

INCLUDING FIELDS USING _SOURCE_INCLUDES

To include a custom list of fields, append the `_source_includes` parameter with comma-separated fields. Say we want to fetch the `title`, `rating`, and `genre` fields from our `movies` index in our response and suppress the others. We can execute the following command.

Listing 5.15 Selectively including few fields

```
GET movies/_doc/3?_source_includes=title,rating,genre
```

This returns the document with these three fields, filtering out the rest:

```
{
  ...
  "_source" : {
    "rating" : 9.3,
    "genre" : "drama",
    "title" : "The Shawshank Redemption"
  }
}
```

This response has both the original document information (under the `_source` object) and the associated metadata. We can also rerun the query using the `_source` endpoint instead of `_doc` to eliminate the metadata and obtain a document with custom fields.

Listing 5.16 Returning selective fields with no metadata

```
GET movies/_source/3?_source_includes=title,rating,genre
```